

Anko 库 核心讲解（Android）

Anko 库 核心讲解（Android）

Anko 是 JetBrains 推出的 **Kotlin 专属 Android 开发库**，核心目标是简化 Android 原生开发的样板代码（如布局编写、Intent 跳转、异步任务、SharedPreferences 操作等），让 Kotlin 代码更简洁、可读性更高。

⚠ 重要说明：Anko 目前已**停止维护**（最后更新 2019 年），官方推荐用 Jetpack Compose/View Binding/ViewModel 等替代，但了解其设计思想仍有价值，且部分老项目仍在使用。

一、核心模块

Anko 分为多个模块，按需引入（Gradle）：

Code block

```
1 // 核心模块（必备）
2 implementation "org.jetbrains.anko:anko-common:0.10.8"
3 // 布局 DSL（替代 XML 布局）
4 implementation "org.jetbrains.anko:anko-sdk27:0.10.8" // 对应 API 27，按需改版本
5 // 布局+控件
6 implementation "org.jetbrains.anko:anko-appcompat-v7:0.10.8"
7 // 异步任务
8 implementation "org.jetbrains.anko:anko-coroutines:0.10.8"
```

二、核心使用场景（最常用）

1. 替代 XML 布局：Anko DSL 编写布局

传统 XML 布局需要单独文件，Anko 可直接在 Kotlin 代码中用 DSL 编写布局，无需 findViewById（自动绑定）。

示例：编写一个简单布局

Code block

```
1 class MainActivity : AppCompatActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
```

```

4
5      // Anko DSL 编写布局 (替代 activity_main.xml)
6      verticalLayout { // 垂直线性布局
7          // 文本控件
8          val tvTitle = textView {
9              text = "Anko 示例"
10             textSize = 20f
11             textColor = Color.BLACK
12             padding = dip(16) // dip 自动适配屏幕
13         }
14
15         // 按钮控件
16         button("点击我") {
17             textSize = 16f
18             margin = dip(8)
19             // 点击事件 (简化)
20             onClick {
21                 toast("按钮被点击了") // Anko 简化 Toast
22                 tvTitle.text = "点击成功!" // 直接操作控件, 无需 findViewById
23             }
24         }
25     }
26 }
27 }

```

核心优势：

- 无需 XML 文件，布局与逻辑内聚；
- 控件自动绑定，省去 `findViewById` /ViewBinding 样板代码；
- 支持链式调用，语法更简洁；
- 尺寸单位（dip/sp）自动适配，无需手动转换。

2. 简化系统 API 调用

Anko 封装了大量 Android 原生繁琐的 API，如 Toast、Intent、SharedPreferences、对话框等。

(1) Toast & Snackbar（简化）

Code block

```

1  // 普通 Toast
2  toast("普通提示")
3  // 长 Toast
4  longToast("长时间提示")
5  // Snackbar (需传入 View)
6  snackbar(findViewById(android.R.id.content), "Snackbar 提示") {

```

```
7     action("确定") { toast("点击了Snackbar按钮") }
8 }
```

(2) Intent 跳转（简化页面跳转/传参）

Code block

```
1  // 基础跳转
2  startActivity<SecondActivity>()
3
4  // 带参数跳转
5  startActivity<SecondActivity>(
6      "name" to "张三",
7      "age" to 20
8  )
9
10 // 跳转并返回结果
11 startActivityForResult<SecondActivity>(REQUEST_CODE, "id" to 100)
```

(3) SharedPreferences（简化）

Code block

```
1  // 获取 SharedPreferences 并操作
2  val sp = defaultSharedPreferences
3  sp.edit {
4      putString("username", "anko_user")
5      putInt("level", 1)
6      apply() // 自动提交
7  }
8
9  // 读取
10 val username = sp.getString("username", "")
```

(4) 对话框（简化 AlertDialog）

Code block

```
1  alert("这是Anko简化的对话框", "标题") {
2      positiveButton("确定") { toast("点击确定") }
3      negativeButton("取消") {}
4  }.show()
5
6  // 列表对话框
7  selector("选择选项", listOf("选项1", "选项2")) { dialog, index ->
```

```
8     toast("选中了${index}")
9 }
```

3. 异步任务（简化 Coroutine/AsyncTask）

Anko 封装了协程/异步任务，替代繁琐的 `AsyncTask`：

Code block

```
1  // 后台执行任务，主线程更新UI
2  doAsync {
3      // 后台线程：模拟耗时操作
4      val result = fetchDataFromNetwork()
5
6      // 切换到主线程更新UI
7      uiThread {
8          textView.text = result
9      }
10 }
```

4. 其他简化：如 Log、资源获取

Code block

```
1  // 简化 Log
2  info("这是Info日志")
3  error("这是Error日志", exception)
4
5  // 简化资源获取
6  val drawable = ctx.drawable(R.drawable.ic_launcher)
7  val color = ctx.color(R.color.white)
```

三、Anko 的优缺点

优点：

1. 极大减少样板代码（如 `findViewById`、`Intent` 构建、异步任务）；
2. Kotlin 友好，DSL 语法符合 Kotlin 风格；
3. 一站式封装，覆盖布局、UI、异步、存储等多个场景。

缺点：

1. **已停止维护**：官方不再更新，存在兼容性风险（高版本 Android 可能适配问题）；
2. 布局写在代码中，失去 XML 布局的可视化编辑、分离优势；
3. 灵活性略低，复杂布局不如 XML 易维护。

四、替代方案（推荐）

由于 Anko 停更，推荐用 Jetpack 组件替代：

- 布局：View Binding / Data Binding / Jetpack Compose；
- 异步：Kotlin Coroutines + Flow；
- UI 简化：自定义扩展函数（如封装 Toast/Dialog/Intent）；
- 存储：DataStore（替代 SharedPreferences）；
- 日志：Timber 等第三方库。

总结

Anko 是 Kotlin 早期简化 Android 开发的经典库，核心价值是用 **Kotlin DSL 和扩展函数消除样板代码**。虽然已停更，但其设计思想（如扩展函数、DSL、简化系统 API）仍值得学习，日常开发中也可借鉴其思路封装自己的工具类。

（注：文档部分内容可能由 AI 生成）